

Bodensee-Sportplatz Image-Service

.NET 9 · Architektur-Ticket mit Go / No-Go-Leitplanken

Next.js/Prisma bleibt Herr über das Haupt-Schema (Plätze, Adressen, Kommentare, Bewertungen, Geodaten). Du baust einen **schlanken, eigenständigen Dienst**, der ausschließlich Bild-Upload und -Verarbeitung übernimmt. Kein Feature-Vollständigkeits-Anspruch – der Dienst soll genau eine Sache können und das robust.

1 Datenbank & Schema-Isolation

EF Core + PostgreSQL, eigenes Schema `images_svc` (nicht `public`) – verhindert Kollisionen mit Prisma-Migrationen in derselben DB.

Entität `SportFieldImage`: `Id`, `SportFieldId` (reiner `int`-Wert, **keine** Navigation Property/Relation zu einer `SportField`-Entität – die kennt dieser Dienst nicht), `FilePath`, `ThumbnailPath`, `Status` (`PendingApproval/Approved`), `CreatedAt`.

GO

- Ein `DbContext`, eine Entität, eine Migration.

NO-GO

- Repository-Pattern/Unit-of-Work über EF Core drüberbauen – EF Core *ist* bereits dein Repository/Unit-of-Work. Diese Zusatzschicht ist ein häufiges Tutorial-Muster aus der EF6-Ära, das heute nur Boilerplate erzeugt.

2 Upload-Endpunkt + Channel-Pipeline

`POST /api/sportsfields/{spotId}/images`: Datei entgegennehmen, Größe + Mime-Type/Magic-Bytes prüfen, Rohdatei temporär ablegen, DB-Eintrag mit `Status = PendingApproval` anlegen.

Bild-ID an einen in-memory `System.Threading.Channels.Channel<T>` übergeben, sofort mit `202 Accepted` antworten – der User wartet nicht auf die Verarbeitung.

GO

- Ein einzelner injizierter `Channel<Guid>` (oder `int`) als Singleton-Service, `Writer` im Endpunkt, `Reader` im Worker.

NO-GO

- Wenn du beim Suchen auf **RabbitMQ, MassTransit, Kafka oder Azure Service Bus** stößt – das sind Lösungen für Kommunikation *zwischen mehreren Prozessen/Servern*. Du hast einen Prozess mit zwei Teilen (API + Worker), die sich In-Memory eine Queue teilen. Kein externer Broker nötig, kein zusätzlicher Container.

3 Background Worker

`BackgroundService`, liest fortlaufend aus dem `Channel<T>`.

Komprimiert zu `.webp`, erstellt Thumbnail, verschiebt Dateien in Ziel-Ordner, setzt `Status = Approved`.

GO

- Eine Klasse, die von `BackgroundService` erbt, in `Program.cs` mit `AddHostedService<T>( )` registriert.

NO-GO

- Kein CQRS, kein MediatR, kein „Command“/„Handler“-Vokabular – das sind Patterns für viele parallele Anwendungsfälle mit Cross-Cutting-Concerns (Logging, Validation-Pipeline, Events). Bei 4 Endpunkten und einem Worker macht das nur unnötige Indirektion. Ruf die Verarbeitungslogik direkt aus einer normalen Service-Klasse (`ImageProcessingService`) auf.
- Kein Domain-Driven-Design-Overkill (Value Objects, Aggregate Roots, Domain Events) – `SportFieldImage` ist eine simple Datenzeile mit Statuswert, keine komplexe Geschäftslogik-Trägerin.

4 Ausgabe & Admin-Freigabe

GET `/api/sportsfields/{spotId}/images`: nur Approved-Bilder zurückgeben.

PATCH `/api/images/{imageId}/approve`: geschützter Admin-Endpunkt.

GO

- Beides als Minimal-API-Endpunkte (`app.MapGet( ... )`, `app.MapPatch( ... )`), direkt gegen den `DbContext` bzw. den Service.

NO-GO

- Kein Controller-Klassen-Stil mischen „nur weil's in einem alten Tutorial so war“ – bleib konsequent bei Minimal API, das hält den Dienst klein und lesbar.

Generelle Leitplanke für's ganze Projekt

Wenn eine Anleitung im Netz von einem dieser Wörter spricht, ohne dass du dir erklären kannst *warum genau hier*, ist es vermutlich nicht für diesen Dienst gedacht:

MediatR · CQRS · Domain Events · Repository Pattern · Unit of Work · Message Broker (RabbitMQ/Kafka/Service Bus) · Microservices-Mesh/API-Gateway · Clean-Architecture-4-Projekte-Setup

Diese Begriffe tauchen in .NET-Tutorials extrem oft auf, weil sie in Enterprise-Kontexten mit vielen Teams/Diensten Sinn ergeben – bei einem Ein-Personen-Dienst mit vier Endpunkten sind sie fast immer der falsche Ratschlag, auch wenn die Tutorials selbst gut sind.

Faustregel beim Copy-Paste-Risiko: Bevor du etwas aus einem gefundenen Snippet übernimmst – frag dich „würde ich das *auch* bauen, wenn niemand außer mir je den Code sieht?“ Wenn die ehrliche Antwort nein ist (weil's nur gut aussieht), gehört's nicht rein.